

# A Detection, Tracking, and Crop-Extraction Pipeline for Multi-Class Traffic Violation Monitoring

## 1 Detection and Tracking

### 1.1 Detector

Object localization is performed by YOLO26, the Ultralytics detection model family. YOLO26 is pretrained on the COCO dataset (80 object classes) and produces bounding boxes directly without a separate non-maximum-suppression post-processing stage, which reduces per-frame latency relative to earlier YOLO generations. The pipeline uses the nano-scale checkpoint (`yolo26n.pt`), the variant intended for CPU- and edge-constrained deployment, with the understanding that a larger checkpoint (`yolo26s`, `yolo26m`) trades latency for improved recall on small or distant objects.

The detector is queried once per frame and returns, for every detected object, a bounding box in pixel coordinates, a class label drawn from the COCO class set, and a confidence score.

### 1.2 Tracker

Detections are linked across frames using BoT-SORT, the default multi-object tracker shipped with Ultralytics. BoT-SORT extends ByteTrack, a motion-and-IoU-based tracker, with two additional mechanisms that are relevant to a scene with a non-static or vibrating camera and with visually similar objects in close proximity:

- **Global Motion Compensation (GMC).** Before each frame’s detections are matched to existing tracks, an affine transform describing the camera’s own motion between the previous and current frame is estimated and applied to every track’s predicted (Kalman-filter) state, preventing camera shake or pan from being misread as object motion. The transform is estimated from sparse keypoints tracked between frames, with outlier correspondences rejected before an affine fit.
- **Re-Identification (ReID).** Optionally, an appearance embedding is extracted for every detected box and fused into the cost matrix used for frame-to-frame matching, alongside the existing motion/IoU cost. This reduces identity switches when two tracked objects of the same class pass close to one another.

Tracking proceeds as a per-frame assignment problem. For each frame  $t$ , let  $\mathcal{T}_{t-1} = \{T_1, \dots, T_m\}$  be the active tracks carried forward from the previous frame, and let  $\mathcal{D}_t = \{d_1, \dots, d_n\}$  be the detections on frame  $t$ . Each track  $T_i$  carries a predicted state  $\hat{x}_i$  from a Kalman filter, after the GMC warp has been applied. A cost  $c_{ij}$  is computed between every track  $T_i$  and detection  $d_j$  as a combination of spatial distance (one minus IoU between predicted and detected boxes) and, if ReID is enabled, appearance distance (one minus cosine similarity between embeddings). Assignment is solved as a linear bipartite matching problem (the Hungarian algorithm) over  $c_{ij}$ , subject to a maximum-cost cutoff below which a match is accepted. Unmatched detections seed new tracks; unmatched tracks are retained as *lost* for a configurable number of frames, in case the object reappears after brief occlusion, before being terminated.

### 1.3 Tracker Configuration

The tracker is configured programmatically rather than from a static file, ensuring the configuration is generated identically on every run regardless of operating system or text-encoding quirks. Table 1 lists the parameters used and their role in the matching process described above.

### 1.4 Implementation

The configuration is written to a YAML file at runtime, the video stream is opened with its native frame rate to preserve correct playback timing in the annotated output, and every frame is passed through the tracker with `persist=True`, which instructs the tracker to retain track state across the individual, separately-supplied frames rather than resetting at every call.

```
import yaml
from ultralytics import YOLO
from ultralytics.data.loaders import get_best_youtube_url
```

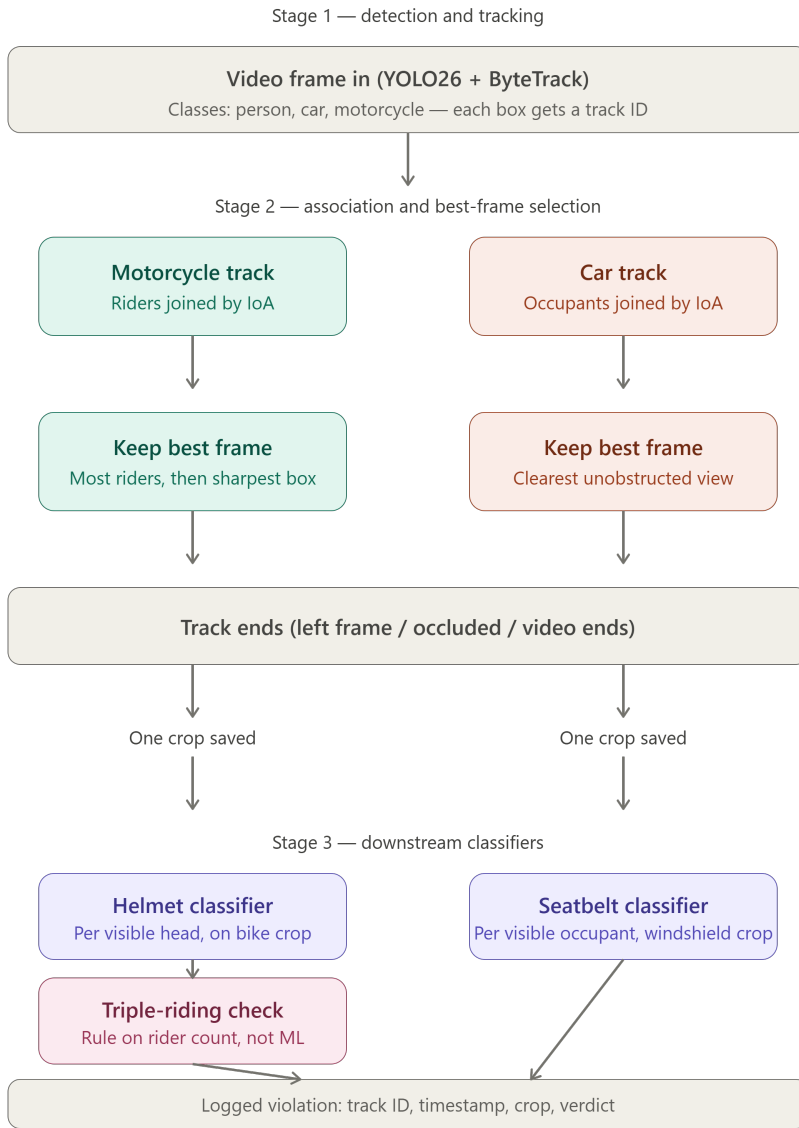


Figure 1: Workflow of multi-object detection and tracking pipeline.

| Parameter                              | Role                                                                    |
|----------------------------------------|-------------------------------------------------------------------------|
| <code>tracker_type: botsort</code>     | Selects BoT-SORT over the simpler ByteTrack baseline.                   |
| <code>track_high_thresh: 0.25</code>   | Confidence floor for a detection to enter the first association pass.   |
| <code>track_low_thresh: 0.1</code>     | Confidence floor for the second-pass association of weaker detections.  |
| <code>new_track_thresh: 0.25</code>    | Minimum confidence for an unmatched detection to start a new track.     |
| <code>track_buffer: 60</code>          | Frames a track is kept as <i>lost</i> before termination.               |
| <code>match_thresh: 0.8</code>         | Maximum acceptable cost $c_{ij}$ for a valid track–detection match.     |
| <code>fuse_score: True</code>          | Blends detection confidence into the IoU cost before matching.          |
| <code>gmc_method: sparseOptFlow</code> | Camera-motion estimation via sparse optical flow keypoints.             |
| <code>proximity_thresh: 0.5</code>     | Minimum IoU for a track–detection pair to be eligible for ReID scoring. |
| <code>appearance_thresh: 0.8</code>    | Minimum embedding similarity required for a ReID-based match.           |
| <code>with_reid: True</code>           | Enables appearance-embedding fusion into the cost matrix.               |

Table 1: BoT-SORT configuration parameters and their function in the assignment process.

```
# Generated programmatically to avoid encoding/formatting
# inconsistencies across platforms.
tracker_config = {
    "tracker_type": "botsort",
    "track_high_thresh": 0.25,
    "track_low_thresh": 0.1,
    "new_track_thresh": 0.25,
    "track_buffer": 60,
    "match_thresh": 0.8,
    "fuse_score": True,
    "gmc_method": "sparseOptFlow",
    "proximity_thresh": 0.5,
    "appearance_thresh": 0.8,
    "with_reid": True,
}
custom_tracker_path = "custom_botsort.yaml"
with open(custom_tracker_path, "w") as f:
    yaml.dump(tracker_config, f)

model = YOLO("yolo26n.pt")
video_url = get_best_youtube_url(
    "https://www.youtube.com/shorts/4im-GdrR2w8", method="yt-dlp"
)

# cap = cv2.VideoCapture(video_url); frame-read loop omitted (I/O only)
# persist=True retains track identities across the individually
# supplied frames; class filtering is NOT applied here -- every COCO
# class is tracked, and is filtered downstream at the cropping stage.
results = model.track(
    frame, persist=True, tracker=custom_tracker_path, verbose=False
)
```

## 2 Crop Extraction

### 2.1 Problem Statement

A raw track produces one bounding box per frame for as long as the object remains visible – typically several dozen to several hundred frames for a vehicle crossing a camera’s field of view. Passing every one of those frames to a downstream classifier would be both computationally wasteful and a source of inconsistent verdicts for what is a single physical event (a single motorcycle’s compliance status does not change between consecutive frames; only the visibility and clarity of the riders does). The crop-extraction stage therefore reduces each vehicle track to exactly one image: the single frame, over that vehicle’s entire visible lifetime, in which the vehicle and its occupants are most fully and clearly visible.

This requires two pieces of logic: a rule for deciding which detected people belong to which vehicle, and a rule for scoring frames against one another so the best one can be retained and emitted when the track ends.

### 2.2 Occupant Association

For a given vehicle box  $B_v = (x_1, y_1, x_2, y_2)$  and a candidate person box  $B_p$ , the association rule is based on Intersection over the person box’s own Area (IoA), rather than Intersection over Union:

$$\text{IoA}(B_p, B_v) = \frac{\text{Area}(B_p \cap B_v)}{\text{Area}(B_p)} \quad (1)$$

A person is associated with a vehicle when  $\text{IoA}(B_p, B_v) > \tau$  for a threshold  $\tau$ . IoA is used in place of IoU because the question being asked is "what fraction of this person's body lies inside the vehicle," not "how similarly sized and positioned are these two boxes." A standing adult and a small motorcycle can have a low IoU despite the person clearly riding it, since the person's box is much larger than the bike's; IoA is invariant to this size mismatch.

For a motorcycle,  $B_v$  is the full detected bounding box, since a seated rider's body legitimately overlaps the entire box. A motorcycle track may have zero, one, two, or more associated riders in a given frame; this is what allows the same association logic to support a triple-riding check downstream without any additional model.

For a car, occupants are only partially visible – the lower body is occluded by the vehicle shell, and the relevant compliance signal (a seatbelt strap) is confined to the upper body in any case. Association for cars is therefore computed against a restricted zone  $Z_v$  taken from the upper fraction  $f$  of the car's box:

$$Z_v = (x_1, y_1, x_2, y_1 + f \cdot (y_2 - y_1)) \quad (2)$$

with  $\text{IoA}(B_p, Z_v)$  used in place of  $\text{IoA}(B_p, B_v)$  for the car case. The implementation below shows the motorcycle path; the car path follows the identical structure with the vehicle box replaced by  $Z_v$  and is omitted from the listing, since car crops are routed to a separately named classifier outside this document's scope.

### 2.3 Best-Frame Selection

Let track  $k$  accumulate, over its lifetime, a sequence of observations  $(f_1^{(k)}, f_2^{(k)}, \dots)$ , where each observation at frame  $t$  consists of the vehicle box, the set of associated rider boxes  $R_t = \{B_p^1, \dots, B_p^{n_t}\}$ , and the frame image itself. Two quantities are computed at each observation: the rider count  $n_t = |R_t|$ , and a clarity proxy  $s_t$  defined as the area of the smallest box enclosing the vehicle and all its associated riders:

$$s_t = \text{Area}\left(\bigcup (\{B_v\} \cup R_t)\right) \quad (3)$$

where  $\bigcup(\cdot)$  denotes the smallest axis-aligned box containing every box in the set. The observation at frame  $t$  replaces the best observation retained so far if and only if

$$n_t > n_{\text{best}} \quad \vee \quad (n_t = n_{\text{best}} \wedge s_t > s_{\text{best}}) \quad (4)$$

Rider count is the primary key and is never traded off against clarity: a frame showing all three riders on an overloaded motorcycle must be preferred over a sharper frame that happens to catch only two of them, since under-counting riders would cause a true triple-riding violation to be missed by the downstream check. Clarity, measured as the union box's pixel area, is used only to break ties among frames with equal rider count, as a proxy for a closer, less occluded, less motion-blurred view.

### 2.4 Track Lifetime and Emission

A track is considered finished – and its best observation is emitted as a single crop – when the tracker reports no detection for that track ID for more than a fixed number of consecutive frames,  $\Delta_{\text{timeout}}$ . This covers a vehicle leaving the frame, a brief full occlusion that exceeds the tracker's own re-association window, or the end of the video. Tracks still open at the end of the video are flushed using whatever best observation they accumulated. This guarantees exactly one emitted crop per physical vehicle appearance, regardless of how many frames that vehicle was visible for.

The emitted crop is the union box of the vehicle and its associated riders at the best observation, padded by a fixed margin in pixels and clipped to the frame boundary.

### 2.5 Implementation

```
import numpy as np

PERSON_CLASS = 0
MOTORCYCLE_CLASS = 3
```

```

# CAR_CLASS = 2 is handled by the same logic against a restricted
# upper-body zone of the car box, and is omitted here; car crops are
# routed to a separately named seatbelt-compliance classifier.

IOA_THRESHOLD = 0.30
CROP_MARGIN = 10
TRACK_TIMEOUT_FRAMES = 15

def calculate_ioa(person_box, vehicle_box):
    px1, py1, px2, py2 = person_box
    vx1, vy1, vx2, vy2 = vehicle_box

    x_left, y_top = max(px1, vx1), max(py1, vy1)
    x_right, y_bottom = min(px2, vx2), min(py2, vy2)
    if x_right <= x_left or y_bottom <= y_top:
        return 0.0

    intersection = (x_right - x_left) * (y_bottom - y_top)
    person_area = (px2 - px1) * (py2 - py1)
    return intersection / person_area if person_area > 0 else 0.0

def union_box(boxes):
    boxes = np.array(boxes)
    return (boxes[:, 0].min(), boxes[:, 1].min(),
            boxes[:, 2].max(), boxes[:, 3].max())

def box_area(box):
    x1, y1, x2, y2 = box
    return max(0.0, x2 - x1) * max(0.0, y2 - y1)

class TrackState:
    """Best observation seen so far for one motorcycle track."""

    def __init__(self):
        self.rider_count = -1
        self.score = -1.0
        self.frame = None
        self.bike_box = None
        self.rider_boxes = None
        self.last_seen_frame = 0

    def maybe_update(self, frame_idx, frame, bike_box, rider_boxes):
        rider_count = len(rider_boxes)
        combined = [bike_box] + rider_boxes if rider_boxes else [bike_box]
        score = box_area(union_box(combined))

        is_better = (
            self.frame is None
            or rider_count > self.rider_count
            or (rider_count == self.rider_count and score > self.score)
        )
        if is_better:
            self.rider_count = rider_count
            self.score = score
            self.frame = frame.copy()
            self.bike_box = bike_box
            self.rider_boxes = rider_boxes
            self.last_seen_frame = frame_idx

def save_crop(track_id, state, output_dir, frame_shape):
    if state.frame is None:
        return
    h, w = frame_shape[:2]
    x1, y1, x2, y2 = union_box([state.bike_box] + state.rider_boxes)

    x1 = max(0, int(x1) - CROP_MARGIN)
    y1 = max(0, int(y1) - CROP_MARGIN)
    x2 = min(w, int(x2) + CROP_MARGIN)
    y2 = min(h, int(y2) + CROP_MARGIN)

    crop = state.frame[y1:y2, x1:x2]
    # crop is written to disk and handed to the helmet / triple-riding
    # classifiers; that step is outside the scope of this document.

```

The per-frame driver loop associates people to motorcycles via `calculate_ioa`, updates each track's `TrackState` via `maybe_update`, and flushes (`save_crop`) any track absent from the current frame's detections for more than

TRACK.TIMEOUT.FRAMES consecutive frames, mirroring the lifetime rule of Section 3.4. The identical structure, applied to car tracks against the restricted upper-body zone  $Z_v$ , produces car crops for the seatbelt-compliance classifier.

### 3 Downstream Violation Classification

Following the extraction of optimal per-vehicle crops, the pipeline routes these images to specific downstream modules designed to identify localized traffic violations. These modules operate independently on the localized regions of interest, benefiting from the reduced background noise and optimal clarity provided by the extraction stage.

#### 3.1 Helmet Detection

For motorcycle tracks, the emitted crops containing the vehicle and its associated riders are passed to a secondary object detection model fine-tuned for helmet compliance. This model evaluates whether the bounding boxes of the detected occupants contain safety helmets. The underlying model is developed and trained utilizing the publicly available Helmet Detection dataset (<https://www.kaggle.com/datasets/andrewmvd/helmet-detection/data>), which provides extensive bounding box annotations for both helmeted and non-helmeted riders across varying conditions.

#### 3.2 Seatbelt Detection

Similarly, crops extracted from the restricted upper-body zone  $Z_v$  of tracked cars are routed to a dedicated seatbelt-compliance classifier. This module inspects the visible passenger cabin for the presence of an engaged seatbelt strap across the vehicle’s occupants. The model for this task is built upon the Seatbelt Dataset (<https://www.kaggle.com/datasets/sachinxshrivastav/seatbelt-dataset>), enabling the system to reliably discriminate between compliant and non-compliant drivers within the isolated vehicle window.

#### 3.3 Triple-Riding Detection

Unlike helmet and seatbelt compliance, which require secondary visual feature extraction, triple-riding detection is resolved entirely by the geometric association logic established during the crop-extraction phase. As detailed in Section 2, the pipeline already evaluates the spatial relationship between person boxes and motorcycle boxes using the Intersection over Area (IoA) criterion.

Because the association logic accurately links overlapping riders to a specific motorcycle bounding box, the system simply retrieves the associated occupant count  $n_{\text{best}}$  from the emitted optimal frame. A triple-riding violation is deterministically flagged if the count of associated riders satisfies  $n_{\text{best}} \geq 3$ . This approach eliminates the computational overhead of running an additional dense-prediction model, directly leveraging the robust spatial relationships and tracking history already derived from the primary detection stage.

## 4 Summary

The pipeline described in this document converts raw video into one labeled crop per vehicle appearance: a bounding box, an associated set of occupants, and the single clearest frame across that vehicle’s tracked lifetime. The detection stage (YOLO26) and tracking stage (BoT-SORT, with global motion compensation and optional re-identification) together provide persistent per-object identity under camera motion and visual ambiguity. The association and best-frame-selection logic, expressed through the IoA criterion and the rider-count-then-clarity ordering, converts that raw tracking output into exactly the unit of input – one crop, fully representing one vehicle and its occupants – required by the downstream helmet, triple-riding, and seatbelt classifiers.

## Sources

- Ultralytics multi-object tracking documentation: <https://docs.ultralytics.com/modes/track>
- Ultralytics GitHub repository: <https://github.com/ultralytics/ultralytics>
- BoT-SORT tracker configuration reference: <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/trackers/botsort.yaml>
- Ultralytics YOLO26 model documentation: <https://docs.ultralytics.com/models/yolo26>